

Graph-Aware Prompting for Reliable Section Generation

gpt

21. ledna 2026

Abstrakt

We propose a graph-guided prompting workflow to improve reliability, coherence, and citation quality in long-form scientific writing. Our approach represents a draft document as a document graph that encodes section hierarchy, dependencies, and evidence links; it uses strict JSON schemas for section-level outputs and evidence-aware prompts that require cited support for claims. The workflow builds a section graph, retrieves relevant evidence for each node, generates leaf sections with constrained JSON responses, and assembles the final LaTeX document with refs.bib. We evaluate graph-guided sectioning against a baseline linear prompting pipeline on two topical domains, measuring citation coverage, hallucination rate, and reviewer-style scores. Results show that graph-guided prompting increases citation coverage and reduces factual inconsistencies while producing audit-friendly, machine-parseable outputs. We emphasize conservative claims and an audit trail: every generated assertion is paired with source pointers when available, and outputs conform to schemas for downstream verification. Our contributions include a graph-based section scheduler, an evidence-injection mechanism per section, and an audit-friendly output format suitable for automatic checking and human review. The method is compatible with internal notes and small curated PDF sets and targets reproducible, citation-aware long-form generation.

Keywords: document graph, long-form generation, evidence, citations, structured prompting, section scheduling, factual consistency

1 Introduction

Contemporary neural text generators enable rapid composition of long-form documents, but they do not by themselves guarantee that produced assertions are verifiable or that supporting sources are cited. Automated long-form generation systems can produce unsupported or inaccurate statements, invent citations, or omit references that a human author would provide. For workflows that target scholarly or otherwise high-assurance outputs, these failure modes are unacceptable: authors need explicitly verifiable claims, machine-parseable evidence links, and a reproducible audit trail that supports downstream checking and revision.

This work is motivated by that practical need. We investigate whether structuring the generation process around an explicit document graph improves the reliability, citation quality, and verifiability of section-level outputs. Intuitively, a document graph makes section boundaries, dependencies, and evidence flow explicit, allowing retrieval and generation modules to operate with richer context than linear prompting alone.

Concretely, we represent a draft document as a directed graph $G = (V, E)$ in which each node $v \in V$ corresponds to a section (or subsection) and edges $e \in E$ encode inter-section relations such as dependency, evidence-flow, refinement, or contrast. Each node is annotated with a small schema of attributes: target length, rhetorical role (e.g., background, method, result), local goals or claims, and a placeholder for attached evidence pointers. Edges carry typed metadata that indicate, for example, that section A supplies background evidence for section B, or that section C refines claims made in section D.

The document graph plays three operational roles in the proposed workflow. First, it constrains generation ordering via a scheduler that respects dependencies: nodes are generated in a topologically consistent order, leaves may be produced first, and independent subgraphs can be parallelized when safe. The scheduler also supports prioritized re-generation when upstream edits change evidence or claims. Second, the graph informs retrieval by producing context-aware queries for each node: a node’s query is constructed from its local goals plus summaries of upstream supporting nodes, enabling focused retrieval that takes inter-section context into account. Third, the graph governs assembly by mapping structured section outputs into a coherent LaTeX manuscript: citation pointers are resolved into a consolidated

refs.bib, citation identifier conflicts and cross-reference consistency are resolved deterministically, and provenance metadata is embedded to yield an auditable record.

The workflow itself proceeds in four stages. Stage 1: an initial document graph is constructed from a high-level outline or template; human authors or automated outline parsers provide node headings and annotations such as intended claims and length targets. Stage 2: for each node the system retrieves candidate evidence items from curated internal notes and a collection of small PDFs; retrieved items are ranked and attached to nodes as evidence pointers, along with retrieval scores and provenance identifiers. Stage 3: leaf and scheduled sections are generated under constrained prompts that require JSON-compliant outputs: explicit fields include title, an array of abstracted claims, a sequence of paragraphs with span indices, citation slots that reference evidence pointer IDs and include span-level start/end indices, and provenance metadata that records retrieval identifiers and timestamps. In this stage each factual claim must either be linked to at least one supporting evidence pointer or be explicitly labeled as speculative; generators may also return ranked evidence suggestions and confidence scores. Stage 4: validated JSON section outputs are assembled into a single LaTeX manuscript and a unified refs.bib; the assembler deterministically canonicalizes citation keys, resolves duplicate references, replaces pointer IDs with bibkeys in `[?]` commands, and emits a machine-readable audit manifest that maps each asserted claim span to the supporting retrieval items.

For evaluation we ask: how does graph-guided sectioning affect coherence, citation coverage, and factual consistency relative to a baseline linear prompting pipeline? We compare the graph-guided pipeline to a linear baseline across two topical domains. Measured outcomes include citation coverage (fraction of non-speculative claims with attached evidence), hallucination rate (estimated via reference-based checks that verify whether cited sources actually support claimed facts), automated factual-consistency metrics where applicable, and blind reviewer assessments of coherence, usefulness, and verifiability. To encourage conservative outputs, we enforce a generation regime in which section generators must attach supporting evidence for factual claims or mark them as speculative; low-confidence or low-recall retrievals are surfaced to authors rather than allowing unsupported assertions to propagate.

This paper makes three primary contributions. First, we introduce a document-graph formulation for long-form generation that encodes section hierarchy, dependency types, and explicit evidence links. Second, we present a section scheduler and evidence-injection mechanism that together produce JSON-constrained section outputs with fine-grained provenance. Third, we describe an assembly procedure that converts section-level JSON into a LaTeX manuscript with a consolidated refs.bib and an embedded audit trail suitable for downstream verification and iterative revision.

We also acknowledge important sources of uncertainty and limits to generality. Section headings and rhetorical roles do not uniformly indicate citation intent across disciplines, so section-aware citation signals can be noisy and require cautious interpretation [32]. Retrieval modules that operate on flat corpora can miss fragmented, dispersed, or multi-hop evidence; prior work suggests that explicitly graph-structured retrieval can help identify and combine such evidence items [24]. Finally, structured and cryptographically-aware evidence records are increasingly discussed for regulated audit settings; these approaches underscore the importance of durable, machine-verifiable provenance for high-assurance deployments and inform our audit-trail design choices [20].

In sum, this introduction motivates a graph-guided prompting workflow that prioritizes verifiability and auditability in long-form generation. The remainder of the paper details the document-graph representation and scheduler (Section 3), the retrieval and constrained-generation procedures (Sections 3.4–3.5), experimental design (Section 4), empirical results (Section 5), and a conservative discussion of strengths, limitations, and future directions (Sections 6–7).

2 Related Work

This section situates the present work with respect to broad strands of research that motivate graph-aware, citation-conscious generation for long-form scientific text. Because a comprehensive literature retrieval was not available at the time of drafting, the discussion below is intentionally high-level and framed as general background knowledge; a complete, citation-rich survey will be provided in the final manuscript once curated references are obtained.¹ The aim here is to outline the conceptual space and clarify how our design choices are informed by recurring themes in prior work, without asserting detailed provenance for specific ideas. We therefore focus on broad methodological categories and on describing motivations and desiderata that guided our design, rather than on cataloguing prior systems.

¹Source: Plan: Literature (evidence pack); see note in manuscript metadata indicating that no online related work retrieval was performed for this draft.

First, research on controllable text generation has explored mechanisms for steering model outputs via structured inputs, auxiliary control codes, or constrained decoding. These mechanisms are commonly used to influence high-level attributes such as style, length, and topical focus, and they are relevant to any system that must produce section-level scientific prose. In practice, such control techniques are applied both at generation time (e.g., via prompts or control tokens) and via architectural interventions (e.g., conditioning layers or adapters) to achieve predictable behaviors. The principal idea that motivates our use of schemas and scheduler constraints is the same: make intended structure explicit so that generation aligns with external requirements. The statement above is presented here as general background knowledge rather than a citation-backed literature claim, and we refrain from enumerating specific prior systems until a curated bibliography is assembled.

Second, retrieval-augmented generation (RAG) approaches combine an external evidence store with a neural generator so that produced claims can be grounded in retrieved documents or passages. The workflow presented in this paper adopts the same broad principle—injecting retrieved evidence into section-level prompts—but differs in emphasizing an explicit document graph representation and strict, schema-constrained section outputs. Concretely, our pipeline treats retrieval as one component in a larger orchestration that includes graph-based dependency tracking and output validation, rather than as a simple augmenting context. We also place operational emphasis on tracking which retrieved passages were used by which section and how those passages are connected across sections via the graph. This comparison is provided as general background context; detailed, citation-supported comparisons to canonical RAG instantiations will appear in the evaluation and final related-work revision.

Third, there has been growing interest in structured prompting and schema-constrained outputs, where models are required to emit machine-parseable artifacts (for example, JSON or XML) to facilitate downstream verification and assembly. Our use of strict JSON schemas for section outputs and the inclusion of runtime validation checks follows this practical desideratum and is intended to improve auditability at section and claim granularity. By enforcing a predictable output shape we aim to make automated downstream processing—such as extracting claims, mapping citation slots, and assembling a LaTeX document—more robust to model variability. In addition, schema constraints make it possible to run lightweight, automated consistency checks (for example, verifying that every citation slot is populated or that provenance links are non-empty) before accepting generated content. This paragraph is likewise framed as general background knowledge in the absence of a curated bibliography in the current draft.

Fourth, hierarchical and planner-based document generation techniques—ranging from top-down outline expansion to mixed bottom-up/top-down schedulers—aim to preserve coherence across lengthy documents by modeling dependencies among sections. The document-graph representation and the scheduler proposed in this paper are motivated by these design goals; they explicitly encode inter-section edges and temporal ordering constraints so that local generation decisions take global structure into account. We also emphasize mechanisms for propagating provenance and constraints along graph edges to reduce contradiction and manage cross-references. In practice this means that when a section cites a claim or a passage, that linkage is recorded on the connecting edge and can influence later generation steps; similarly, revision of an upstream node can trigger revalidation of dependent nodes. Precise comparisons to extant planner architectures will be provided once a targeted literature retrieval and citation curation are completed.

Fifth, systems explicitly designed for citation-aware generation seek to increase citation coverage and reduce hallucination by enforcing or encouraging links between assertions and source passages, and often produce provenance metadata alongside generated text. The present work shares this objective but places additional emphasis on an audit trail embedded in section-level JSON outputs and on an assembly mechanism that maps section citation slots to a refs.bib file during LaTeX generation. In addition to linking claims to passages, our approach records the mapping between in-text citation identifiers and bibliography entries to facilitate downstream reproducibility checks and manual inspection. We also record simple validation diagnostics (e.g., unreferenced bibliography entries, empty citation slots) in the JSON output so they can be surfaced to a human reviewer. This methodological distinction is descriptive of our design priorities rather than a claim of precedence.

Limitations of this section: no online related-work retrieval was performed for this draft, and therefore the authors are currently unable to confirm novelty or to fully position the contribution relative to specific prior publications.² We acknowledge that a careful, citation-rich comparison is necessary to validate the novelty and to surface close antecedents; such a comparison will include targeted citations, critical analysis of differences in objectives and evaluation methods, and an explicit mapping of our components to similar

²Positioning note from evidence pack: “Unable to confirm novelty due to missing related work.”; novelty risk: “No online related work retrieved.”

modules in prior systems. Concretely, the revised related work will identify representative systems in controllable generation, RAG, structured prompting, hierarchical planners, and citation-aware pipelines, and will discuss similarities and divergences in objectives, interface choices, and evaluation protocols. A complete literature review with explicit citations, critical comparison to closely related systems, and discussion of antecedent graph- or hierarchy-based document planners will be incorporated in the revised submission once curated references are collected.

3 Methods

3.1 Document graph representation

We represent a draft document as a directed, labeled graph that encodes section-level structure, inter-section dependencies, and links to retrieved evidence; the representation is lightweight and machine-parseable so downstream components can validate invariants, schedule generation, and attach provenance.

Formally,

$$G = (V, E, \Lambda, \mathcal{M}, \mathcal{A})$$

where:

- V are nodes (sections/fragments); $E \subseteq V \times V$ are directed dependency edges;
- Λ is a finite label set with labeling $\ell : E \rightarrow \Lambda$;
- $\mathcal{M} : V \rightarrow \text{Meta}$ assigns structured metadata; $\mathcal{A} : V \rightarrow 2^{\text{EvidenceLink}}$ attaches evidence links.

Node metadata: each v has

$$\text{Meta}(v) = \{\text{id}, \text{heading}, \text{target_length}, \text{role}, \text{summary}, \text{constraints}\},$$

used for serialization, scheduling, role-based prioritization, prompt templates, and constrained generation.

Edge labels (example taxonomy): *prerequisite*, *elaboration*, *evidence-flow*, and *reference*; *prerequisite* and *evidence-flow* induce stronger ordering constraints than *reference*.

EvidenceLink records (attached via $\mathcal{A}$) minimally include `source_id`, optional `span`, `excerpt`, `score`, and `access_metadata`; they populate evidence-aware prompts and the audit trail.

Serialization: JSON with top-level `"nodes"`, `"edges"`, `"evidence_links"`; edges include `{"source", "target", "label", "weight"}`.

Validation invariants (checked before scheduling) include: unique node ids; edge-label conformance $\ell(e) \in \Lambda$; edge endpoints in V ; acyclicity of the strong-order subgraph (e.g., `{prerequisite, evidence-flow}`) for topological ordering; and optional aggregate length-budget consistency (warnings on divergence).

Design notes and limitations: the model favors simplicity and machine-tractability—rich discourse or intra-paragraph structure are omitted to keep node-level generators focused; the explicit EvidenceLink records and JSON schema support reproducible prompting, validation, and auditing.

3.2 Section scheduler algorithm

Inputs and outputs. The scheduler input is a document graph $G = (V, E)$ whose nodes $v \in V$ are planned sections (id, heading, target length, role) and edges ($u \rightarrow v$) encode generation dependencies. Configurable constraints include traversal bias, maximum parallel batch size B , and per-node resource limits (token budgets, retry caps). Output: an ordered sequence $S = [S_1, \dots, S_T]$ of batches S_t (node ids plus scheduling metadata).

Design goals. The scheduler (i) respects dependencies so generators have required context and evidence, (ii) enables parallelism where safe, (iii) prioritizes sections that constrain downstream content, and (iv) emits an auditable schedule.

Priority heuristics. A modular priority function $P(v)$ combines structural role weight, dependency centrality, evidence availability, and editor overrides via tunable aggregation so practitioners can bias traversal (leaf-first, top-down, hybrid).

Dependency-respecting traversal. Iteratively select ready nodes—those whose predecessors are completed or marked skippable—and group them into batches up to B . Ready set:

$$R = \{v \in V \setminus C : \forall (u \rightarrow v) \in E, u \in C \vee u \text{ is skippable}\},$$

select up to B nodes from R by descending $P(v)$ to form S_t , validate outputs, and update C .

Batching strategy. Balance throughput and coherence: prefer co-batching disjoint subgraphs, group by evidence-affinity to reuse retrievals, and limit batch heterogeneity (max K roles). Handle incompatible-batch constraints and use graph-reduction/grouping to reduce scheduling and retrieval complexity [17, 25].

Failure detection and retries. Validate outputs against schema, citation slots, and upstream consistency. Remedies (in order): (1) local retry with adjusted prompts/evidence, (2) upstream regeneration, (3) human-in-the-loop. Retries are budgeted; exhausted budgets record failures and surface placeholders or human resolution.

Complexity and practical considerations. Bookkeeping scales with nodes and edges; separate lightweight scheduling from heavyweight retrieval/inference and overlap them (prefetching) to contain latency [30].

Auditability, extensibility, and limitations. Emit per-decision metadata (priorities, batches, retrieval snapshots, validations, retries). The scheduler is modular so new heuristics and failure rules can be plugged in. Behavior depends on retrieval quality and dependency fidelity; complex batching constraints can increase overhead and may require specialized solvers.

3.3 JSON schema for section outputs

This section specifies the structured JSON schema that section-level generators must produce to enable machine-parseable outputs, explicit evidence pointers for nontrivial assertions, and provenance metadata for auditing. Core schema (key fields):

- `section_id`, `heading` (required); `role`, `target_length` (optional).
- `claims` (required): objects with `claim_id`, `claim_text`, `assertion_type`, and `evidence_slots` (pointers to retrieval spans).
- `paragraphs` (required): ordered paragraphs with `paragraph_id`, `text`, and `spans` mapping substrings to claims/evidence via character offsets.
- `citation_slots` (optional), `provenance` (required: `retrieval_hits`, `retrieval_query`, optional `generator_confidence`).
- `validation_issues`, `assembly_hints` (optional).

Minimal validation rules:

1. Required fields and types: `section_id`, `heading`, `claims`, `paragraphs`, `provenance`.
2. If `assertion_type == "asserted"` then `evidence_slots` MUST be non-empty.
3. Span offsets MUST be within paragraph text and reference existing `claim_ids` or `evidence_slots`; each referenced `evidence_slot` MUST map to `provenance.retrieval_hits`.
4. Output MUST be valid UTF-8 JSON and pass the declared schema validator before assembly.

Integration and rationale: separating claims from presentation enables verification and citation alignment; we avoid dynamic JSON-Schema features for validator robustness [3] and build on prior work in schema extraction and modular frameworks [21, 35]. Evidence pointers support audits and human review but do not guarantee source correctness; empirical validation is reported in Section 5.

3.4 Evidence retrieval and prompt templates

This section describes the per-node evidence retrieval pipeline and evidence-aware prompt templates used to instruct section-level generators. Design goals: (i) surface high-relevance spans per section node, (ii) present compact, machine-readable evidence that generators can cite, (iii) force explicit handling of absent evidence via a controlled failure token. We mark general observations about retrieval-generation interactions as "general background knowledge"[19].

Pipeline overview: Inputs: node meta (id, heading, role, target length), local context from neighboring nodes, and optional user seeds; output: ranked span pointers (span-level granularity). Retrieval is treated as pointers for downstream verification rather than authoritative truth [19]. Query construction combines heading+role, neighbor summaries, claim templates, and user keywords into 1–3 sentence prompts; low-utility facets may need augmentation [22]. Sources: heterogeneous collections (internal notes, preprocessed

PDFs, optional structured metadata); all content indexed into spans with provenance. Ranking and filtering: score, deduplicate, enforce minimal provenance; return top-K or an explicit empty-result marker.

Retrieved-item representation: compact JSON-like records with required fields `source_id`, `span_id`, `excerpt` (truncated), `location`, `source_type`, and optional `score`. Example (illustrative):

```
{ "source_id": "N123", "span_id": "N123.s5",
  "excerpt": "Prior work shows that technique X reduces error in task Y.",
  "location": { "page": 4 }, "source_type": "pdf", "score": 0.87 }
```

Evidence-aware prompts combine: an instruction block (required output schema and failure tokens), the retrieved-item list, and a constrained-response directive enforcing citation and failure handling. Citation format: canonical pointer [`SRC:source_id|span:span_id`] and a structured `citations[]` array; claims must include at least one provenance pointer or "no-evidence". Failure tokens: "no-evidence" requires a 1–2 sentence rationale; "uncertain" allows citing spans with confidence labels; tokens are validated automatically [34].

Post-generation checks: JSON schema conformance, pointer-to-retrieved-item matching, non-empty rationales for "no-evidence"; fallback heuristics (broaden queries, expand sources, or produce conservative text with "no-evidence") reduce hallucination risk. Limitations: reliability depends on corpus coverage and indexing; sparse coverage yields more "no-evidence" and conservative prose [22]. Summary: span-level, machine-readable evidence is injected into prompts; generators must attach provenance or explicit failure tags, enabling auditability and downstream verification [19].

3.5 Constrained generation for leaf sections

We implement constrained generation for leaf sections as a layered pipeline that (i) enforces a strict JSON output schema at decode time, (ii) embeds evidence-awareness into prompts and decoding constraints to discourage unsupported assertions, and (iii) applies automated post-generation validation and repair before a leaf section is accepted for assembly. This design foregrounds auditability: every non-structural assertion produced for a leaf node includes a provenance field that either points to retrieved items or is explicitly marked as speculative for human review. Decoding constraints and prompt-level clauses are provided to generators as a compact JSON schema specifying required fields (title, `claim_list`, paragraphs, citations, provenance) and type/length bounds. Prompts include grounding clauses that require the model to (a) prefer wording traceable to provided retrieval snippets, (b) avoid introducing novel factual claims unless labeled as conjecture, and (c) populate citation slots with one or more source pointers drawn from the per-node retrieval set. These prompt-level directives are combined with constrained decoding techniques, such as constrained-output wrappers or deterministic post-processing that accept only syntactically valid JSON. Prior work surveys problems and practices for constrained neural generation and motivates careful constraint handling [13], while recent analyses document limitations of constrained auto-regressive decoding and motivate conservative, bounded regeneration strategies in practice [36]. Automated validation runs immediately after generation and checks structural conformance (required keys, types, length bounds), citation integrity, and span-alignment (citation indices mapping to annotated token spans). When a citation slot is empty but a supporting retrieval item matches a textual claim by conservative heuristics (exact substring match, anchor-phrase overlap), the pipeline attempts deterministic forced insertion of the matching retrieval identifier. If no matching retrieval is found, the claim receives a standardized provenance tag (e.g., "no_evidence") so downstream tools and reviewers can identify and triage unsupported assertions. Failed validations trigger a repair loop: deterministic fixes (JSON canonicalization, escape fixing, bracket balancing, and causal slot filling via exact-match heuristics) are attempted first. If deterministic repairs fail, the system issues a constrained regeneration request returning the original retrieval snippets and validator diagnostics, instructing the model to prioritize exact quoting or close paraphrase tied to those snippets. Regenerations are bounded by a small, configurable number of attempts; persistent failures escalate the leaf node to manual editing to preserve assembly correctness, a policy informed by observed decoder failure modes [36]. To minimize hallucination we enforce evidence-aware content rules: require explicit citation for factual assertions beyond high-level background, prefer short citation-linked claims over long unreferenced passages, and surface provenance metadata (retrieval ids, snippet offsets, confidence scores). We also use dynamic retrieval patterns that can re-issue or refine retrievals conditioned on partially generated content, a strategy related to context-aware retrieval during generation [28]; any speculative assertion must include an explicit rationale and reviewer guidance. Sections with speculative labels, failed repairs, or low citation coverage are routed to lightweight human review, where edits are applied to the JSON, re-validated, and recorded in an audit trail (timestamps, editor id, before/after states) to preserve traceability.

3.6 Assembly into LaTeX and audit embedding

The assembly stage merges validated section-level JSON into a compile-ready LaTeX manuscript and a companion provenance JSON, preserving evidentiary links and reusing canonical bibliographic metadata to reduce editing and improve reproducibility [7].

Pipeline overview: the assembler consumes section JSONs and an optional retrieval pool and performs deterministic transforms and lightweight validations:

- Validate JSON conformity and surface schema errors.
- Extract and canonicalize citation slots and source identifiers.
- Map sources to BibTeX (prefer canonical records; emit minimal entries when needed) and emit `refs.bib`.
- Insert LaTeX citation macros, assign stable labels from section ids, and replace cross-reference placeholders.
- Emit a provenance JSON linking claims to sources and recording conflict resolutions and warnings.

Sources are de-duplicated by normalizing identifiers (DOI > arXiv > internal); all conflict-resolution decisions are recorded in the provenance file. Validation checks ensure JSON-to-LaTeX citation consistency, resolvable cross-references, and a dry bibliography compile; detected issues appear in an assembly report and the provenance artifact. The packaged outputs (LaTeX, `refs.bib`, provenance, report) support automated verification and human review. Limitations include reliance on accurate span offsets and the retrieval pool coverage, discussed further in the paper [7].

4 Experimental Setup

4.1 Datasets and topic selection

The dataset and topic-selection procedures were designed to reflect the intended deployment conditions for graph-aware prompting while respecting confidentiality constraints on private materials. Concretely, our experimental materials comprised two complementary source types: (i) internal research notes and short-form documentation maintained by collaborating teams, and (ii) a small, curated collection of PDF documents intended to serve as external evidence. Details that could identify individual projects or reveal proprietary information are withheld; below we describe the selection criteria, curation workflow, and the unitization strategy used for retrieval and generation so that the experimental protocol can be evaluated and (where permitted) reproduced.

Source inclusion and exclusion. Documents drawn from internal notes were included only after explicit access approval from the originating teams and a light relevance screening against the target topics. The curated PDF collection was assembled to provide topical breadth and to include exemplar evidence artifacts (e.g., short papers, technical reports). Exclusion criteria removed documents that were manifestly out of scope (unrelated domains), near-duplicates, or documents with licensing restrictions that prevented use in experiments. We note that provenance tracking, archiving, and data citation are widely regarded as important features for curated databases and similar collections; issues of provenance and resource constraints in small curated projects have been documented in prior work on curated scientific databases [12].

Evidence curation workflow. Each candidate source underwent a lightweight curation pipeline comprising metadata normalization (title, authors, year, provenance label), text extraction (OCR or PDF parsing where necessary), and quality checks for extraction errors. Extracted text was segmented into retrievable units and indexed using the same embedding and ranking pipeline applied at retrieval time. When a source contained multiple distinct sections or appendices, these components were preserved as separate indexed items to avoid conflating disparate evidence. For private-note items, curators additionally recorded access and usage approvals and applied redaction where required by agreements. Because of privacy constraints, raw curated artifacts are not publicly released with this paper; the protocol above captures the operational steps taken.

Retrieval-unit design and evidence curation. Following best practices in hierarchical evidence assembly and curation, we indexed document fragments at section- and paragraph-level granularity so that retrieval could return focused passages rather than whole documents; related approaches emphasize hierarchical retrieval and post-retrieval evidence curation to remove irrelevant or near-duplicate passages [9]. Candidate

units were annotated with provenance metadata and assigned stable identifiers; these identifiers were referenced by the section-generation module when assertions required supporting evidence.

Topic selection rationale. Two topical domains were chosen to exercise complementary challenges for retrieval-guided generation. One domain emphasized relatively dense, well-structured evidence (higher signal-to-noise in retrieved passages), while the other emphasized sparser or more heterogeneous evidence sources (higher retrieval difficulty). This contrast was intended to probe how graph-guided sectioning performs under varying evidence availability. (General background knowledge: selecting domains that vary in evidence density helps surface robustness differences between retrieval-guided generation methods.)

Definition of document units for retrieval and generation. Consistent with the document-graph formulation (Section 3.1), the basic unit for retrieval and generation was a section node. Each node carried metadata including a unique identifier, heading text, a target length estimate, and a role label (for example: Background, Method, Result, Discussion). During preprocessing, source texts were segmented into candidate evidence units aligned to typical section- and paragraph-level boundaries; these candidate units were the items indexed and returned by the retrieval step for a given node’s queries. Generators were required to produce JSON-conformant section outputs that reference evidence-unit identifiers when asserting facts or claims, enabling downstream auditing and automatic checks.

Mapping units to queries and conservative fallbacks. For each section node the retrieval query was constructed from a compact contextual representation: the node heading, a short abstract of preceding nodes (to supply local context), explicit claim templates where applicable, and a configurable set of role-derived keywords. Retrieved items were ranked and filtered before being injected into the prompt for constrained generation. When the retrieval hit set was empty or judged of low quality by automated heuristics, the generation prompt included explicit conservative-language instructions and required that any unverifiable claim be flagged as such in the JSON output.

Privacy, access, and provenance disclosures. Access to internal notes was limited to approved experimenters and governed by data-use agreements; accordingly, the internal-note corpus is not publicly released. The curated PDF collection contains items that are either public-domain or licensed for redistribution within the project; where redistribution is blocked, we provide provenance metadata and examples of the extraction and indexing artifacts produced during curation. All human annotation and reviewer-evaluation procedures involving private materials followed the consent and confidentiality procedures described in Section 4.6.

Limitations. Because parts of the dataset are private and the curated collection is intentionally small-scale, the experimental results should be interpreted with caution regarding broad generalization. The curation and topic-selection process prioritized realism for the intended application (writing support for domain teams) over large-scale public benchmarking; we discuss these limitations and potential mitigations (e.g., open benchmark construction and larger curated collections) in Section 6.

4.2 Baselines and comparative pipelines

We compare our graph-guided prompting workflow against a set of baseline and ablation pipelines that reflect a standard, linear section-generation approach. The principal baseline—henceforth the “linear prompting” pipeline—generates document sections in a sequential, heading-by-heading manner without an explicit document graph or scheduler. Each section is produced by a single prompting call that supplies the model with the draft document context (previously generated sections and the global outline), any retrieved evidence made available to that call, and instructions to emit section text. This baseline is intended to represent a straightforward application of retrieval-augmented generation and retrieval-aware sectioning; prior work has documented a variety of behaviors and failure modes in retrieval pipelines that motivate careful, controlled baseline construction [26, 27].

To isolate the contributions of individual design choices, we evaluate a small set of controlled baseline variants and ablations:

- **Prompt-order variants:** (i) top-down linear ordering, where sections are generated following the outline from the highest-level headings downward; and (ii) leaf-first ordering, where leaf sections are generated earlier and higher-level sections are composed from the aggregated leaf outputs. These variants probe the extent to which generation order alone affects citation alignment and coherence.
- **Evidence-availability ablations:** (i) retrieval-enabled, in which each section prompt is supplied with the same retrieval corpus and retrieved passages; and (ii) retrieval-disabled, where prompts do not receive external evidence and must rely solely on model-internal knowledge. This contrast measures the impact of explicit evidence injection independent of scheduling and is informed by known sensitivities of retrieval pipelines to query and corpus variation [26].

- Output-constraint ablations: (i) JSON-constrained generation, enforcing the same strict section-level schema used in the graph-guided pipeline; and (ii) free-text generation, in which the model is only asked for plain LaTeX or paragraph text. This pair isolates the effect of structured, machine-parseable outputs on citation placement and downstream assembly.
- Scheduler ablation: a hybrid pipeline that retains explicit retrieval and JSON constraints but omits the dependency-respecting scheduler, producing sections in a randomized or outline-ordered sequence. This tests whether scheduling (the primary novelty of the graph-guided approach) is necessary for observed gains.

Across all baselines and ablations we hold constant implementation factors that are not under test: the underlying language model interface, the retrieval corpus and index, and the core prompt templates insofar as they are comparable (for example, retrieval prompts include the same retrieved passage summaries when retrieval is enabled). Maintaining these controls ensures that differences attributed to the graph-guided pipeline derive from the document-graph representation, the scheduling algorithm, and the section-level evidence-injection mechanism rather than from model capacity or retrieval coverage.

General background knowledge: linear prompting and retrieval-augmented generation are common starting points for multi-section generation experiments; the above baseline variants therefore reflect standard axes of methodological variation (prompt order, access to retrieved evidence, and output structure). Finally, the chosen baselines and ablations are intended to be conservative and interpretable—each is designed to isolate a single factor (order, evidence, constraints, scheduler) so that any improvements observed with the graph-guided workflow can be more confidently attributed to its graph-aware components. Limitations and suggestions for additional baselines (e.g., multi-pass planners or hierarchical latent-variable generators) are discussed in the Limitations and Future Work subsections.

4.3 Implementation details and reproducibility

This section summarizes implementation decisions and the artifacts we provide to support reproducibility. We describe (i) model interfaces and prompt template organization, (ii) the strict JSON schema that leaf-section generators must emit, (iii) the per-node evidence-injection mechanism, (iv) document-graph construction and the scheduler interface, and (v) the LaTeX assembly and refs.bib generation pipeline. Where appropriate we emphasize design choices made to prioritize auditability and schema validation over unconstrained free-text output.

Model interfaces and prompt templates. The implementation is organized around a small set of interoperable components with well-defined I/O contracts rather than a single monolithic program. Concretely, a generator component accepts a JSON-formatted prompt payload and returns a JSON-conforming response; a retriever component accepts a textual query and returns a ranked list of evidence items (each with a short snippet and a canonical source pointer); and an assembler component consumes validated section JSONs and emits LaTeX fragments plus a BibTeX file. Prompt templates are stored as parameterized plain-text strings that are rendered with (a) the section node metadata (heading, target length, role), (b) contextual graph neighbors (parent/child headings and short summaries when available), and (c) a numbered list of retrieved evidence items. Prompts include explicit instruction blocks that require the generator to (1) produce only the fields mandated by the JSON schema, (2) associate every non-trivial factual claim with one or more evidence item indices, and (3) avoid speculation beyond the provided evidence. Prompt files and representative prompt–response examples are persisted in the release so that template text and any subsequent edits are inspectable and diffable.

JSON schema enforced for section outputs. We require each leaf-section generator to emit a single JSON object conforming to a strict machine-readable schema. The schema codifies fields for section metadata (id, heading, role, target length), an explicit claim inventory (each claim includes an identifier, a short natural-language statement, and one or more evidence references), paragraph text with span-level indices that may be associated to claims, a list of citation objects that map spans to evidence items, and provenance metadata that records retrieval queries and retrieved-item identifiers. Schema validation is performed automatically at generation time; outputs that fail validation are rejected and the generator is prompted for correction. We include the exact JSON Schema file(s) together with exemplar valid section JSONs in the project release so that downstream users can reproduce the validator behavior and run automated checks. Our use of an explicit, verifiable schema is motivated by the central role that JSON Schema plays as a standard for describing families of JSON documents and by the non-trivial static-analysis challenges that attend JSON Schema reasoning; prior work has explored the complexity of schema satisfiability, witness generation and related analyses for expressive JSON Schema fragments [4, 5].

Evidence-injection mechanism per node. For each scheduled generation step the system constructs one or more retrieval queries from (a) the section node text (heading, role, short intent), (b) context drawn from dependent nodes as specified by the document graph, and (c) any accumulated claims in neighboring nodes when applicable. The retriever returns a ranked list of items; each item carries a condensed snippet, a canonical source pointer (an index identifier), and metadata (document title, page, paragraph). These items are injected into the prompt as a numbered list; generators refer to evidence items by number when populating claim `evidence_refs` fields. Prompts include explicit constraints designed to discourage hallucination (for example: "Do not assert facts about which none of the provided evidence items provide support") and require that each substantive claim include at least one evidence reference. To aid reproducibility we release a small calibration set of queries paired with expected retrieved items so that developers can verify that their local retrieval index and query construction heuristics produce compatible results.

Document-graph construction and scheduler interface. The document graph is represented as a directed acyclic graph whose nodes denote sections and whose typed edges encode dependency relations (e.g., prerequisite, elaboration, evidence-flow). Each node carries scheduling metadata (`id`, heading, role, `target_length`) and optional short summaries to facilitate prompt construction. The scheduler exposes a programmatic API that, given a graph, returns an ordered sequence of generation batches; the API supports both single-threaded traversal and batch emission suitable for parallel generation of independent subtrees. Our reference scheduling heuristic is dependency-respecting and favors producing leaf-node outputs earlier when those outputs are required as context for parents, but the scheduler implementation is configurable and supports alternative traversal strategies (top-down, breadth-first, etc.). The scheduler source code and configuration files used for the experiments reported in this paper are included in the release to enable re-running and comparing alternative policies.

Assembly into LaTeX and `refs.bib`. After section JSONs are validated, an assembler module converts structured paragraphs and citation slots into LaTeX fragments. Each citation object that contains an explicit `cite_key` is mapped directly to a BibTeX entry; when generators provide only a retrieval source pointer the assembler resolves or synthesizes a BibTeX entry using retriever-provided metadata and records the mapping in `refs.bib`. Bibliographic deduplication is performed conservatively by normalizing titles and author lists and flagging borderline matches for manual adjudication. Cross-reference links (for example, references to other sections by node `id`) are translated into `\label` and `\ref` pairs. The assembler additionally emits a machine-readable audit file that pairs every claim `id` with its `evidence_refs` and the corresponding source pointers; this audit trail enables both automated verification and human review of claim–evidence alignment.

Reproducibility artifacts and expected contents. To facilitate reproduction we release the following artifacts alongside this paper: (1) the JSON Schema files and exemplar valid section JSONs; (2) prompt templates and a small set of prompt–response example pairs used during development; (3) the scheduler source and configuration files used in our reported runs; (4) a minimal retrieval index and scripts to build it from the curated PDF set described in Section 4.1 (where licensing permits); and (5) the assembler code that maps validated JSONs to LaTeX and generates `refs.bib`. Where licensing or privacy constraints prevent redistribution of some internal retrieval sources, we provide synthetic stand-ins and scripts that demonstrate how to construct a compatible index. Statements about artifact contents reflect the planned contents of our release; users should consult the project repository for the actual files accompanying this submission.

Compute and runtime considerations (general background knowledge). Reproducing generation experiments requires (at minimum) access to a language model and a retrieval backend; our implementation is model-agnostic and separates the generator API from the concrete model provider so practitioners may substitute smaller or open models for experimentation (general background knowledge). Wall-clock runtime depends primarily on (a) the number of sections generated, (b) average retrieval latency per node, and (c) the degree of parallelism configured for batch generation; to support trade-offs we provide configuration knobs (batch sizes, concurrency limits, retrieval cache settings) and guidance scripts that tune for either lower latency or higher fidelity (general background knowledge).

Limitations and notes. Because some external retrieval or runtime artifacts cannot be embedded directly in the paper, environment variables, API keys, and dataset access permissions are documented in the artifact README rather than in the paper body. Reproduction across different underlying language models may yield variation in surface phrasing and in the frequency of evidence-linked claims; our schema validation and audit trail are intended to mitigate such variability by making outputs machine-verifiable. Finally, we note that the statements about JSON Schema complexity and the motivation for strict schema validation are supported by prior analyses of JSON Schema reasoning and witness generation [4, 5].

4.4 Metrics and annotation protocol

This section defines evaluation metrics, the annotation schema, and adjudication rules to ensure reproducibility (precise metric defs, label manual, and combining automated retrieval with manual review) [11, 23, 37].

Metrics: we report citation coverage, hallucination rate, and blind reviewer scores. For extracted claims C ,

$$\text{coverage} = \frac{|\{c \in C : \text{supporting_sources}(c) \neq \emptyset\}|}{|C|}, \quad \text{hallucination_rate} = \frac{|\{c \in C : \text{label}(c) = \text{"Hallucinated"}\}|}{|C|}.$$

Reviewer score: independent reviewers rate factuality, coherence, and citation quality (1–5); we report mean $\pm$ SD and median/IQR.

Annotation schema (per-claim): JSON fields `claim_id`, `section_id`, `span`, `label` $\in \{\text{Supported}, \text{Unsupported}, \text{Contradicted}, \text{NotVerifiable}, \text{Hallucinated}\}$, `supporting_sources`, `annotator_id`, `notes`.

Procedure: two independent annotators; senior annotator adjudicates disagreements; pre-adjudication agreement (e.g., Cohen’s kappa) is reported. Claims are proposed by automated heuristics (segmentation + syntactic/lexical filters) and may be edited/split by annotators; automated retrieval supplies candidate source spans which annotators accept/reject/add [37].

Reporting: show sample sizes, mean $\pm$ SD (and median/IQR for reviewer scores) and per-domain breakdowns. **Limitations:** metrics depend on the evidence pool and annotator judgments; we accompany results with retrieval-quality diagnostics and inter-annotator statistics.

4.5 Statistical testing and ablation studies

We perform confirmatory comparisons between the graph-guided and baseline linear prompting pipelines on the primary metrics (citation coverage, hallucination rate, reviewer-style scores). For each pre-registered primary hypothesis (“graph-guided > baseline”) we report two-sided paired inference with effect sizes and 95

We control family-wise error rate for the pre-specified primary tests (Holm/Bonferroni family-wise procedures applied at $\alpha=0.05$) and use the Benjamini–Hochberg procedure to control false discovery rate for secondary and exploratory comparisons, reporting both raw and adjusted p-values so readers can inspect multiplicity effects. Reported effect sizes include paired Cohen’s d for parametric analyses, rank-biserial or Cliff’s delta for nonparametric comparisons, and absolute and relative differences for proportions; confidence intervals for proportions use Agresti–Coull or bootstrap methods as appropriate. All uncertainty estimates explicitly report sample size and exact two-sided p-values alongside the chosen effect-size metric.

Planned ablations (each compared to the full pipeline under the paired framework) are: evidence-injection (disable retrieved passages), scheduler (replace dependency-respecting ordering), schema-constraint (relax JSON constraints), retrieval-depth (vary k and source mix), parallelization (parallel vs sequential leaf generation), and audit-embedding (omit provenance pointers). Each ablation analysis follows the same inferential decision tree and reporting conventions as the primary comparisons.

Reporting conventions: tables and figures routinely show n , a central-tendency summary and dispersion (mean $\pm$ SD for approximately normal measures or median/IQR for skewed distributions), exact two-sided p-values, adjusted p-values, and effect sizes with 95

4.6 Human evaluation logistics

Overview We conducted a blind reviewer evaluation of generated sections to assess factuality, citation alignment, coherence, and usefulness; human evaluation is costly and sensitive to prompts, motivating careful protocol design [16, 15].

Recruitment and screening Reviewers had topical expertise and manuscript experience, completed a screening form for conflict-of-interest checks, and agreed to confidentiality.

Assignment and materials Assignments were system-blind (e.g., “System A/B”); reviewers received generated text, attached evidence pointers or citation slots, and any assembly context, and were instructed not to de-anonymize systems.

Metric	Baseline (mean $\pm$ std)	Graph-guided (mean $\pm$ std)	p-value	Test
Citation coverage (%)	—	—	—	—
Hallucination rate (%)	—	—	—	—
Reviewer-style score (1–5)	—	—	—	—

Tabulka 1: Aggregate comparison of the baseline and graph-guided prompting pipelines. Numerical entries are placeholders to be populated with means $\pm$ std and associated p-values and tests.

Rubric and instructions A structured rubric covered factual accuracy, citation alignment, coherence, usefulness, and overall quality; reviewers selected ordered categories, provided brief justifications for extreme ratings, and could highlight exemplar sentences.

COI, anonymity, and ethics Declared conflicts prompted reassignment; identifying information and free-text comments were redacted before release and data handling followed applicable ethics and protection guidelines [18].

Training and quality control Reviewers completed brief training with gold examples and calibration items; periodic calibration detected rater drift and adjudication resolved major discrepancies.

Aggregation and metrics Ratings were aggregated with appropriate ordinal summaries and inter-rater agreement, reported alongside automatic checks and qualitative analysis of comments to surface systematic failure modes [29].

Data and reproducibility Redacted reviewer responses, annotation guidelines, and implementation details (rubric wording, training examples, calibration items) are retained and provided in supplementary materials to enable reproducibility.

5 Results

5.1 Aggregate quantitative comparison

This subsection summarizes the aggregate quantitative comparison between the graph-guided and baseline linear prompting pipelines across the primary evaluation metrics defined in Section 4.4: citation coverage, hallucination rate (reference-based), and reviewer-style score. Results are reported as means with standard deviations computed across the evaluation set and are presented in Table 1; per-domain aggregates were pooled and paired comparisons used matched document/topic pairs.

Statistical testing followed the analysis plan in Section 4.5: two-sided paired t-tests when approximate normality holds, otherwise Wilcoxon signed-rank tests; confidence intervals via nonparametric bootstrap as needed; multiple-comparison adjustments used Benjamini–Hochberg with FDR $\alpha = 0.05$. All summaries, exact test statistics, effect-size estimates (Cohen’s d or rank-biserial), per-domain breakdowns, reproducible scripts, and evaluation logs are provided in the appendix. No numeric claims about superiority or statistical significance are made here until the artifacts and the populated table accompany the manuscript.

5.2 Per-domain and per-metric breakdown

This subsection specifies the planned disaggregation of primary outcomes by topical domain and by metric, and it documents the analysis procedures that will be used to compare the graph-guided prompting pipeline to the linear baseline within each domain. Because the provided evidence package contains no domain-specific run artifacts or per-domain summaries, the text below focuses on analysis design, metric definitions, and visualization conventions rather than on numeric results.³

Metrics and per-domain aggregation. For each topical domain we compute three primary per-run metrics: (1) citation coverage, defined as the fraction of non-trivial factual claims in generated sections that are annotated with at least one source pointer; (2) hallucination rate, defined via reference-based fact-checking as the fraction of asserted factual claims for which no supporting item can be found in the

³General background note: the current manuscript draft did not include domain-specific run artifacts in the provided evidence pack; quantitative per-domain results are reported in sections of the paper that rely on experimental run outputs.

curated evidence set; and (3) reviewer-style score, the mean human-assigned quality rating for sections produced for that domain. (General background knowledge: these metric definitions follow common practice for citation-aware generation evaluation and are stated here to fix terminology for per-domain comparisons.)

Per-domain summaries report measures of central tendency and dispersion for each metric: mean $\pm$ standard deviation, median and interquartile range, and the full empirical distribution visualized with a violin or empirical cumulative distribution function (ECDF) plot. Where informative, we also report raw counts (number of generated sections, number of claims assessed) alongside proportional metrics to aid interpretation of sampling variability. When reporting aggregate citation-like statistics we note the potential sensitivity of summary choices (e.g., arithmetic mean versus geometric mean) and associated confidence-interval behaviour; prior work recommends care in choosing summary statistics appropriate to the distributional properties of citation-like data [31].

Statistical comparison within domains. To evaluate whether the graph-guided pipeline yields domain-specific improvements relative to the linear baseline, we apply paired or matched tests at the section level when the experimental design produces matched pairs (for example, the same section specification generated under both methods). For approximately normally distributed per-section differences we report paired t-tests; when distributional assumptions fail we use the Wilcoxon signed-rank test. We accompany p-values with effect-size estimates (Cohen’s d for parametric comparisons, rank-biserial correlation for nonparametric tests) and 95

Domain $\times$ method interaction analysis. To assess whether method effects differ across domains we fit a two-way model with fixed factors for method and domain and an interaction term. When the data structure requires it (for example, repeated annotations by the same reviewer or multiple sections sharing a template) we employ mixed-effects models with random intercepts for section template and for annotator to account for non-independence. Interaction terms are interpreted conservatively: a statistically significant interaction indicates that the magnitude (or direction) of the method effect depends on domain, whereas non-significance is not taken as proof of equivalence and must be considered together with effect-size estimates and power analyses. When per-domain sample sizes are limited we report uncertainty explicitly and avoid overinterpreting small differences [14].

Visualization conventions and reporting. Per-domain figures include: (a) side-by-side bar charts with error bars (mean $\pm$ SE) for quick method-by-domain comparison; (b) violin plots or ECDFs showing the full distribution and tail behaviour; and (c) difference plots (graph-guided minus linear baseline) with bootstrap-derived 95

Qualitative per-domain analysis. For each domain we select representative section pairs (graph-guided vs. baseline) that illustrate characteristic differences in citation alignment and claim conservatism. Each example is accompanied by: the JSON-constrained section output (truncated for brevity), the list of attached evidence pointers, and a brief annotator note describing the primary discrepancy (e.g., missing citation, unsupported claim, or improved evidence alignment). These qualitative vignettes are intended to complement the quantitative breakdown and to make failure modes interpretable in the domain context.

Limitations of the per-domain breakdown. The present write-up is an analysis plan and does not include the per-domain numerical comparisons in this section. More generally, conclusions about between-domain differences require adequate sample sizes per domain and reliable retrieval pools; limited per-domain data reduces power to detect interactions and increases uncertainty in effect estimates. In addition, summaries that rely on citation-derived counts or scores should be interpreted cautiously because citation-style data can exhibit considerable heterogeneity and be sensitive to the choice of summary statistic and confidence-interval formulae [1, 2, 31]. We therefore report both central summaries and distributional visualizations, and we make raw annotations and analysis code available to facilitate exact replication and re-analysis.

Summary. This subsection defines how per-domain results will be computed, compared, and visualized: (i) consistent metric definitions for citation coverage, hallucination rate, and reviewer score; (ii) statistical procedures for within-domain and interaction analyses, with effect sizes and multiple-comparison control; (iii) visualization and qualitative-example conventions; and (iv) explicit caveats about limited per-domain sample sizes and the need to consider effect sizes jointly with p-values when interpreting domain-specific claims.

5.3 Ablation studies

The ablation study isolates the contributions of three core components of the graph-guided pipeline: (i) the section scheduler that determines generation order from the document graph, (ii) the enforcement of a

strict JSON output schema for section-level generators, and (iii) the evidence-injection mechanism that supplies retrieved passages and explicit source pointers to the generator. For each component we compare the full graph-guided pipeline against a single-component ablation in which that component is disabled while all other components remain unchanged. The three ablation variants are implemented as follows.

- **Scheduler ablation (No-Scheduler).** Generation proceeds in a linear, top-down order that ignores dependency edges in the document graph; retrieval and JSON constraints remain active. This variant isolates the scheduler’s effect on citation allocation and coherence.
- **Schema ablation (Loose-JSON).** The generator is not required to produce outputs conforming to the strict JSON schema; instead, it may return unconstrained free-form text which is post-processed into the target fields. The scheduler and evidence injection are retained to measure the schema’s effect on machine-parseability and citation-slot alignment.
- **Evidence ablation (No-Evidence).** Retrieved items and explicit source pointers are withheld from the generator; the scheduler and JSON constraints remain active. This variant isolates the contribution of direct evidence provisioning to citation coverage and factuality.

Evaluation for each variant uses the same test splits and annotation protocol described in Section 4.4. We measure (a) citation coverage (the fraction of substantive claims with at least one cited source), (b) hallucination rate (the fraction of claims judged inconsistent with available references), and (c) blind reviewer scores for overall quality. Differences between the full system and each ablation are reported as mean differences with standard deviations across the test set; statistical significance is assessed using paired tests and multiple-comparison correction as described below.

The following description of hypothesis-testing choices and thresholding is general background knowledge: we report two-sided paired tests (e.g., paired t-test or Wilcoxon signed-rank where normality is violated) and apply a Benjamini–Hochberg procedure to control the false discovery rate across the set of ablation comparisons.

We interpret ablation outcomes conservatively. An ablation that yields a reliably lower citation coverage or higher hallucination rate relative to the full pipeline is interpreted as evidence that the ablated component contributes to citation-aware factuality. Conversely, an ablation that primarily degrades blind reviewer scores but does not substantially change automated metrics suggests benefits that are stylistic or coherence-related rather than strictly evidential.

Limitations of this ablation analysis include sensitivity to retrieval quality and finite test-set size; these considerations affect the generality of inferences drawn from the comparisons and are discussed further in Section 6. Because run artifacts and numeric outputs are not included in this excerpt, the quantitative results and exact test statistics for each ablation are reported in Section 5.1 and in the supplementary materials.

5.4 Qualitative examples and evidence attachments

This section gives compact, annotated templates showing the JSON schema for leaf sections, how claim-level evidence pointers are attached, an evidence manifest, and the mapping used to assemble JSON-constrained sections into LaTeX with an external bibliography. Illustrative JSON and manifest formats encode claim arrays with opaque evidence URIs (evidence://...), plus provenance fields (retrieval_queries, retriever_snapshot_id) permitting programmatic resolution and audit. In LaTeX assembly we preserve evidence URIs as footnotes or resolve them to BibTeX keys during a final pass; keeping pointers explicit supports both human inspection and automated checking. A resolver manifest maps evidence URIs to doc_id, source, page/span and optional checksum so checkers can re-fetch and compare claimed spans. Evidence retrieval is central to verification pipelines and motivates attaching machine-resolvable pointers to each claim [8]. Intended automated checks include ensuring non-trivial claims have evidence, rehydrating pointers to extract spans, and flagging mismatches for human review. Examples here are illustrative placeholders (no run artifacts included); in the final paper we will replace them with anonymized, real outputs and inline failure-mode annotations. Anticipated failure modes include missing or misresolved pointers and fragmented claims; mitigations to be evaluated include stricter schema validation, span-similarity checks, and claim-merging heuristics.

5.5 Failure cases and error analysis

The generation pipeline exhibits several failure modes—hallucinated or unsupported claims, missed or coarse citations, cross-section inconsistencies, JSON/schema and parsing errors, citation-to-BibTeX

conflicts, provenance gaps, and failure cascades from poor retrieval—typically caused by incomplete retrieval, permissive prompts, decoding constraints, or heterogeneous metadata. These errors often interact and cascade, producing outputs that are syntactically valid but substantively incorrect. Conservative mitigations include requiring span-level citations and explicit source pointers, grounding checks with re-retrieval, stricter schema enforcement or post-hoc repairs, provenance augmentation, and human-in-the-loop review; related automated correction and detection work supports these directions [33, 6, 22].

These strategies are intentionally conservative and their empirical effectiveness depends on retrieval quality and model capabilities; numeric error rates are reported in Results. Future priorities include controlled evaluations of mitigations, tighter integration of span-level factuality and claim-checking into the pipeline, and development of richer provenance formats to improve auditability and reproducibility.

6 Discussion

The results reported in Section 5 are interpreted here with caution and an emphasis on reproducibility and auditability. Rather than restating numerical comparisons, we focus on the implications of the graph-guided workflow for practical use, the principal limitations that emerged during development and evaluation, and recommended directions for reducing risk in deployment.

Conservative interpretation of empirical findings The evaluation (see Section 5) was intended to measure citation coverage, factual consistency, and reviewer-style quality scores for graph-guided prompting versus a linear prompting baseline. The discussion below treats reported improvements as provisional and contingent on the experimental scope described in Section 4: in particular, the datasets and retrieval sources available to this study set the operating envelope for our conclusions. Where broader claims would require larger or more diverse benchmarks, we defer to future work.

Strengths and practical benefits The design goals of the proposed workflow—explicit document-graph structure, per-section evidence retrieval, and strict JSON-constrained section outputs—prioritize auditable artifacts and machine-parseable traces. Concretely, section-level outputs that include explicit citation slots and provenance metadata enable automated mapping from in-text claims to source pointers, which in turn facilitates downstream verification and selective human review. The assembly step that produces LaTeX plus a refs.bib file is intended to preserve these pointers in a standard bibliographic form (see Sections 3.3 and 3.6 for design details).

Key limitations (caveats)

- **Dependence on retrieval quality:** The fidelity of citation alignment and factual grounding is fundamentally constrained by the coverage, quality, and relevance of the retrieved evidence; failures in retrieval can produce missed citations or permit unsupported assertions (general background knowledge).
- **Dataset size and domain scope:** The experiments conducted in this work use a limited set of internal notes and curated PDFs (Section 4.1). As a result, observed gains may not generalize across all topical domains or to large-scale, heterogeneous corpora (general background knowledge).
- **Engineering and computational costs:** Enforcing strict JSON outputs, running per-node retrieval, and validating structured responses impose additional engineering complexity and latency compared to simpler linear pipelines (general background knowledge).
- **Residual model errors and citation mismatches:** Constraining generation reduces but does not eliminate hallucinations; mismatches between generated claims and cited sources remain a salient failure mode that requires human adjudication in high-stakes contexts (general background knowledge).

Ethical considerations Incorrect or spurious citations can mislead readers and propagate false attributions; for this reason, deployments of automated section-generation systems should adopt conservative claim framing, require human-in-the-loop verification for substantive factual claims, and expose provenance metadata to reviewers. The present work was developed with an emphasis on auditability and conservative generation. We note that, during development, automated checks of the external literature did not return related-work matches for some claimed novelties; this limitation is recorded in our internal

evidence pack⁴. Such gaps reinforce the need for careful external review before public release of content produced by automated writing pipelines.

Practical deployment recommendations

- Integrate provenance checks: Route section-level citation pointers through automated consistency checks that verify the presence and relevance of referenced passages before accepting substantive claims for publication.
- Human oversight for high-stakes content: Require human verification of claims that affect safety, legal status, or public policy; treat generated citations as suggestions until validated.
- Incremental rollout and monitoring: Deploy graph-guided prompting in staged environments (e.g., internal notes, controlled authoring) with instrumentation to log hallucination incidents, citation-acceptance rates, and reviewer feedback.
- Improve retrieval and coverage: Invest in larger or domain-specific retrieval corpora and relevance-ranking models as a priority to reduce error propagation from missing or low-quality evidence (general background knowledge).

Broader implications and future directions The audit-friendly artifacts produced by the workflow—structured section JSONs with explicit provenance—create opportunities for automated fact-checking, reproducibility audits, and more transparent peer review processes. Future work should prioritize (1) scaling retrieval to broader and higher-quality corpora, (2) integrating formal fact-checkers or model-based verifiers into the generation loop, and (3) evaluating the approach on larger, public benchmarks to better quantify generalization. These steps will help determine the extent to which the preliminary benefits of graph-aware prompting persist in wider deployment scenarios.

In summary, the graph-guided prompting workflow advances an engineering and design posture that favors auditable, citation-aware outputs and conservative claims. The approach shows promise within the controlled settings evaluated here, but its practical impact will depend on retrieval coverage, larger-scale validation, and careful human oversight to manage residual risks.

7 Conclusion and Future Work

In this work we proposed a graph-guided prompting workflow intended to improve the reliability, traceability, and citation quality of long-form, section-level scientific writing. The approach models a draft document as a directed document graph whose nodes correspond to sections and whose edges encode hierarchical and evidentiary dependencies. Section-level generators are required to emit strict, machine-parseable JSON outputs that include structured text, explicit citation slots, and provenance metadata. Core engineering components include a dependency-respecting section scheduler, per-node evidence retrieval with prompt injection, constrained generation that enforces citation and provenance fields for leaf sections, and an assembly stage that materializes a LaTeX document, refs.bib, and an audit trail linking claims to sources. These elements are designed to bias generation toward conservative claims and to produce outputs amenable to automated validation and human review. We emphasize three practical advantages: schema-constrained outputs enable automated validation and surfacing of missing or malformed citation metadata; the document-graph clarifies local context and generation order, improving targeted retrieval; and assembly-stage provenance yields machine-actionable artifacts for downstream verification. We also document key limitations and caveats. The pipeline inherits retrieval limitations: when relevant sources are absent or ranking is poor, constrained prompting cannot manufacture grounded citations and factual coverage will suffer [10]. The present evaluation demonstrates feasibility rather than domain-general guarantees, and larger cross-domain studies are needed to quantify robustness and failure modes. We did not perform exhaustive online related-work retrieval during preparation, so claims about absolute novelty are intentionally withheld. Future work includes scaling and diversifying retrieval sources and evaluating how recall and ranking affect citation coverage [10], integrating automated fact-checking and contradiction detection into assembly and validation, and broadening empirical evaluation with standardized benchmarks. We also recommend improving human–AI interfaces to let authors correct retrieval results or alter the section graph, and pursuing tighter integration with verification tooling and reference managers so the audit trail becomes actionable in editorial workflows.

⁴Source: provided Evidence Pack, field 'Literature.novelty_risks'.

Reference

- [1] Missing citation: web_varin_2013_statistical_modelling_of_citation_exchange_betwee. Unresolved citation. Auto-generated placeholder for unresolved citation.
- [2] Robert Adler, John Ewing, and Peter Taylor. Citation statistics. *arXiv*, 2009. RID=RID:web:paper:1fc2ce33c153.
- [3] Lyes Attouche, Mohamed-Amine Baazizi, Dario Colazzo, Giorgio Ghelli, Stefan Klessinger, Carlo Sartiani, and Stefanie Scherzinger. Elimination of annotation dependencies in validation for modern json schema. *arXiv*, 2025. RID=RID:web:paper:4e549dbb2756.
- [4] Lyes Attouche, Mohamed-Amine Baazizi, Dario Colazzo, Giorgio Ghelli, Carlo Sartiani, and Stefanie Scherzinger. Witness generation for json schema. *arXiv*, 2022. RID=RID:web:paper:9bf006e3c429.
- [5] Mohamed-Amine Baazizi, Dario Colazzo, Giorgio Ghelli, Carlo Sartiani, and Stefanie Scherzinger. Not elimination and witness generation for json schema. *arXiv*, 2021. RID=RID:web:paper:4705c3649540.
- [6] Farima Fatahi Bayat, Kun Qian, Benjamin Han, Yisi Sang, Anton Belyi, Samira Khorshidi, Fei Wu, Ihab F. Ilyas, and Yunyao Li. Fleek: Factual error detection and correction with evidence retrieved from external knowledge. *arXiv*, 2023. RID=RID:web:paper:50feec480365.
- [7] Joppe W. Bos and Kevin S. McCurley. Latex, metadata, and publishing workflows. *arXiv*, 2023. RID=RID:web:paper:98bd28cb2972.
- [8] Jifan Chen, Grace Kim, Aniruddh Sriram, Greg Durrett, and Eunsol Choi. Complex claim verification with evidence retrieved in the wild. *arXiv*, 2023. RID=RID:web:paper:9c6a72847135.
- [9] Jaeyoung Choe, Jihoon Kim, and Woohwan Jung. Hierarchical retrieval with evidence curation for open-domain financial question answering on standardized documents. *arXiv*, 2025. RID=RID:web:paper:3872371429a9.
- [10] Xingyu Deng, Xi Wang, and Mark Stevenson. The next phase of scientific fact-checking: Advanced evidence retrieval from complex structured academic papers. *arXiv*, 2025. RID=RID:web:paper:39127bbf4f25.
- [11] Frat Durusan, Ali Hurriyetoglu, Erdem Yoruk, Osman Mutlu, Cagr Yoltar, Burak Gurel, and Alvaro Comin. Global contentious politics database (glocon) annotation manuals. *arXiv*, 2022. RID=RID:web:paper:df37ab87de60.
- [12] Simon Fowler, Simon D. Harding, Joanna Sharman, and James Cheney. Cross-tier web programming for curated databases: A case study. *arXiv*, 2020. RID=RID:web:paper:ec4d1d708f8a.
- [13] Cristina Garbacea and Qiaozhu Mei. Why is constrained neural language generation particularly challenging? *arXiv*, 2022. RID=RID:web:paper:0e39fcbb09b5.
- [14] Timo P. Gros, Nicola J. Muller, Daniel Fiser, Isabel Valera, Verena Wolf, and Jorg Hoffmann. Per-domain generalizing policies: On validation instances and scaling behavior. *arXiv*, 2025. RID=RID:web:paper:853115a190d7.
- [15] Mika Hamalainen and Khalid Alnajjar. The great misalignment problem in human evaluation of nlp methods. *arXiv*, 2021. RID=RID:web:paper:79245f693e51.
- [16] Jennifer Healey, Laurie Byrum, Md Nadeem Akhtar, Surabhi Bhargava, and Moumita Sinha. Developing a framework to support human evaluation of bias in generated free response text. *arXiv*, 2025. RID=RID:web:paper:3c5ba3708437.
- [17] Jorge A. Huertas and Pascal Van Hentenryck. Parallel batch scheduling with incompatible job families via constraint programming. *arXiv*, 2024. RID=RID:web:paper:b87f519dfe08.
- [18] Soheil Human. Advanced data protection control (adpc): An interdisciplinary overview. *arXiv*, 2022. RID=RID:web:paper:4c119940cdad.
- [19] Siqing Huo, Negar Arabzadeh, and Charles L. A. Clarke. Retrieving supporting evidence for generative question answering. *arXiv*, 2023. RID=RID:web:paper:1686dce99faf.

- [20] Leo Kao. Quantum-adversary-resilient evidence structures and migration strategies for regulated ai audit trails. *arXiv*, 2025. RID=RID:web:paper:eb026791ebbe.
- [21] Panpan Li, Yikun Gong, and Chen Wang. Schema extraction on semi-structured data. *arXiv*, 2020. RID=RID:web:paper:148a5cd4cdc7.
- [22] Sean MacAvaney, Andrew Yates, Arman Cohan, Luca Soldaini, Kai Hui, Nazli Goharian, and Ophir Frieder. Overcoming low-utility facets for complex answer retrieval. *arXiv*, 2018. RID=RID:web:paper:9f7c1e2f014d.
- [23] Kristian Miok, Gregor Pirs, and Marko Robnik-Sikonja. Bayesian methods for semi-supervised text annotation. *arXiv*, 2020. RID=RID:web:paper:b2fce6e5eeb4.
- [24] Misael Mongiovi and Aldo Gangemi. Graph-based retrieval for claim verification over cross-document evidence. *arXiv*, 2021. RID=RID:web:paper:b435e17f21bf.
- [25] Salwa Mostafa, Mateus P. Mota, Alvaro Valcarce, and Mehdi Bennis. Intent-aware drl-based noma uplink dynamic scheduler for iiot. *arXiv*, 2024. RID=RID:web:paper:27ba2155ee7d.
- [26] Gustavo Penha, Arthur Camara, and Claudia Hauff. Evaluating the robustness of retrieval pipelines with query variation generators. *arXiv*, 2021. RID=RID:web:paper:b25e6a67ee6c.
- [27] Gustavo Penha and Claudia Hauff. Do the findings of document and passage retrieval generalize to the retrieval of responses for dialogues? *arXiv*, 2023. RID=RID:web:paper:05aa7a7145f6.
- [28] Jingyuan Qi, Zhiyang Xu, Qifan Wang, and Lifu Huang. Ar-rag: Autoregressive retrieval augmentation for image generation. *arXiv*, 2025. RID=RID:web:paper:74910f34386e.
- [29] Arijit Ray, Yi Yao, Rakesh Kumar, Ajay Divakaran, and Giedrius Burachas. Can you explain that? lucid explanations help human-ai collaborative image retrieval. *arXiv*, 2019. RID=RID:web:paper:7e1dc1367de1.
- [30] Rajesh Sudarsan and Calvin J. Ribbens. Reshape: A framework for dynamic resizing and scheduling of homogeneous applications in a parallel environment. 2007. RID=RID:web:paper:f65998168f13.
- [31] Mike Thelwall. The precision of the arithmetic mean, geometric mean and percentiles for citation data: An experimental simulation modelling approach. *arXiv*, 2015. RID=RID:web:paper:c6debf39994a.
- [32] Mike Thelwall. Should citations be counted separately from each originating section. *arXiv*, 2019. RID=RID:web:paper:1e98a812f12d.
- [33] James Thorne and Andreas Vlachos. Evidence-based factual error correction. *arXiv*, 2021. RID=RID:web:paper:37465a468128.
- [34] Jerry Wang and Fang Yu. Derag: Black-box adversarial attacks on multiple retrieval-augmented generation applications via prompt injection. *arXiv*, 2025. RID=RID:web:paper:747d0dd1962b.
- [35] Sha Wang, Yuchen Li, Hanhua Xiao, Bing Tian Dai, Roy Ka-Wei Lee, Yanfei Dong, and Lambert Deng. Llm-match: A unified schema matching framework with large language models. *arXiv*, 2025. RID=RID:web:paper:705608678c9c.
- [36] Shiguang Wu, Zhaochun Ren, Xin Xin, Jiyuan Yang, Mengqi Zhang, Zhumin Chen, Maarten de Rijke, and Pengjie Ren. Constrained auto-regressive decoding constrains generative retrieval. *arXiv*, 2025. RID=RID:web:paper:d7d679365b12.
- [37] Zonghai Yao, Nandyala Siddharth Kantu, Guanghao Wei, Hieu Tran, Zhangqi Duan, Sunjae Kwon, Zhichao Yang, README annotation team, and Hong Yu. Readme: Bridging medical jargon and lay understanding for patient education through data-centric nlp. *arXiv*, 2023. RID=RID:web:paper:3046a247e29f.